

Day 3: TypeScript Essentials (Part 1)

Intern: Penumuru Madhu Sudhan Reddy

Topic: Static Typing, Interfaces vs Types, Generics, and Discriminated Unions

Target: Beginner-Friendly & Practical

Overview: JavaScript is dynamically typed — types are checked only when your code runs, which means typos and missing properties cause crashes in production. TypeScript adds a **static type system** that checks your code at compile time in your editor, catching bugs before they ever reach production.

1. Why TypeScript? The 3 Core Benefits

- Catch Bugs at Compile Time:** If you misspell a property (`user.emial` instead of `user.email`) or pass a string where a number is expected, TypeScript gives you an instant red underline error before you even run the code.
- Self-Documenting Code:** When you read a function header, you know exactly what arguments it expects and what it returns without reading the entire function body.
- IDE Superpowers:** Full autocomplete (IntelliSense), instant refactoring (rename a property across 50 files with F2), and automatic error detection in VS Code.

2. Type Annotations & Primitive Types

Type annotations use a colon `:` type after a variable or parameter name:

BASIC TYPE ANNOTATIONS

```
// Primitives:
const projectName: string = "Sobha Ivory";
const totalFloors: number = 24;
const isCertified: boolean = true;
const createdAt: Date = new Date();

// Arrays:
const towerNames: string[] = ["Tower A", "Tower B"];
const floorScores: number[] = [85, 92, 78];

// Function signatures (typed parameters and return type):
function calculateProgress(completed: number, total: number): number {
  if (total === 0) return 0;
  return Math.round((completed / total) * 100);
}

console.log("Progress:", calculateProgress(12, 24) + "%");
```

Terminal Output:
Progress: 50%

3. interface vs type (The NirmanIQ Convention)

Both interface and type can describe the shape of an object. In professional codebases, we follow a clean convention:

Construct	When to Use	NirmanIQ Rule	Example
interface	Describing the shape of objects and domain entities.	MANDATORY for all data models (Project, Tower, Floor).	interface Project { id: string; name: string; }
type	Unions, aliases, primitives, and function types.	MANDATORY for status lists and fixed option sets.	type Status = 'planned' 'in_progress' 'completed';

MODELING CONSTRUCTION ENTITIES (EXERCISE 1)

```
// 1. Use 'type' for discrete status unions:
export type ConstructionStatus = 'planned' | 'in_progress' | 'completed';

// 2. Use 'interface' for entity shapes:
export interface Floor {
  floorNumber: number;
  status: ConstructionStatus;
  engineerName: string;
  notes?: string; // Optional property (has '?') - may be omitted
}

const f1: Floor = {
  floorNumber: 3,
  status: 'in_progress',
  engineerName: 'Suresh Kumar'
  // notes is optional, so it is safe to omit
};

console.log(`Floor ${f1.floorNumber}: ${f1.status}`);
```

Terminal Output:

```
Floor 3: in_progress
```

4. Union Types, Literal Types & Type Narrowing

- **Literal Types:** Limit a value to exact allowed strings: type Priority = 'low' | 'medium' | 'high';
- **Union Types:** Allow a value to be one of several types using |: let id: string | number;
- **Type Narrowing:** Using typeof to check the type inside an if block so TypeScript knows which one it is:

TYPE NARROWING WITH TYPEOF

```
function formatTaskId(id: string | number): string {
  if (typeof id === 'string') {
    return id.toUpperCase(); // TS knows id is a string here
  } else {
    return `TASK-00${id}`; // TS knows id is a number here
  }
}

console.log(formatTaskId("site-a")); // SITE-A
console.log(formatTaskId(15)); // TASK-0015
```

Terminal Output:

```
SITE-A
TASK-0015
```

5. Generics: Reusable, Type-Safe Functions & Classes

Generics allow you to write code that works with **any type** while keeping complete type safety. Think of <T> as a type placeholder (type variable):

GENERIC FUNCTION EXAMPLE

```
// captures whatever type the caller passes in:
function getFirstItem<T>(items: T[]): T {
  return items[0];
}

const num = getFirstItem([10, 20, 30]); // TypeScript infers num is number
const name = getFirstItem(["Tower A", "B"]); // TypeScript infers name is string

console.log("First number:", num);
console.log("First tower:", name.toUpperCase());
```

Terminal Output:

```
First number: 10
First tower: TOWER A
```

The Generic Type-Safe Cache Class (Exercise 3)

Store different data models in a cache while preserving full autocomplete and type checking on retrieval:

GENERIC CACHE STORAGE

```
export class TypedCache {
  private store = new Map<string, unknown>();

  set<T>(key: string, value: T): void {
    this.store.set(key, value);
  }

  get<T>(key: string): T | undefined {
    return this.store.get(key) as T | undefined;
  }
}

interface WorkerProfile { name: string; trade: string; }

const cache = new TypedCache();
cache.set<WorkerProfile>("worker_1", { name: "Ramesh", trade: "Carpentry" });

const worker = cache.get<WorkerProfile>("worker_1");
console.log("Retrieved Worker:", worker?.name, "-", worker?.trade);
```

Terminal Output:

Retrieved Worker: Ramesh - Carpentry

6. unknown vs any: Why any is Banned

The Problem with any:

Using any completely turns off TypeScript's type checker for that variable. It allows you to call non-existent methods and bypass type checks, which leads directly to runtime crashes. At NirmanIQ, any is banned in production code.

Type	What It Means	Can You Access Properties?	Safety
any (Banned)	"I don't care about types. Turn off checking."	Yes, anything allowed without checks (dangerous!).	Zero safety.
unknown (Recommended)	"We don't know the type yet (e.g. from raw API)."	No. TypeScript forces you to check the type first.	100% safe.

SAFELY NARROWING UNKNOWN

```
function processRawData(data: unknown) {
  // data.trim(); // Error! TypeScript prevents calling methods on unknown

  if (typeof data === 'string') {
    console.log("Clean string:", data.trim()); // Safe!
  } else if (typeof data === 'number') {
    console.log("Formatted number:", data.toFixed(2)); // Safe!
  }
}

processRawData("  Sobha Project  ");
processRawData(125000.5);
```

Terminal Output:

Clean string: Sobha Project
Formatted number: 125000.50

7. Discriminated Unions & Exhaustive Checking (State Machines)

A **Discriminated Union** is a set of interfaces that share a single common literal property (the "discriminant", e.g. status). It is the gold standard for modeling state machines in software:

MODELING CONSTRUCTION TASK LIFECYCLE (EXERCISE 4)

```
export interface PendingTask {
  status: 'pending';
  title: string;
  createdAt: Date;
}

export interface InProgressTask {
  status: 'in_progress';
  title: string;
  assignee: string; // Only exists when in progress!
}

export interface CompletedTask {
  status: 'completed';
  title: string;
  reviewer: string; // Only exists when completed!
  completedAt: Date;
}

export type ConstructionTask = PendingTask | InProgressTask | CompletedTask;

// Exhaustive pattern matching:
export function getTaskSummary(task: ConstructionTask): string {
  switch (task.status) {
    case 'pending':
      return `[Pending] ${task.title}`;
    case 'in_progress':
      return `[In Progress] ${task.title} (Assigned to: ${task.assignee})`;
    case 'completed':
      return `[Completed] ${task.title} (Approved by: ${task.reviewer})`;
    default: {
      // If a developer adds a 4th state ('cancelled') but forgets to handle it,
      // this line causes a COMPILE-TIME ERROR:
      const _exhaustiveCheck: never = task;
      return _exhaustiveCheck;
    }
  }
}

console.log(getTaskSummary({ status: 'pending', title: 'Foundation Casting', createdAt: new Date() }));
console.log(getTaskSummary({ status: 'in_progress', title: 'Brickwork', assignee: 'Kiran Rao' }));
```

Terminal Output:

```
[Pending] Foundation Casting
[In Progress] Brickwork (Assigned to: Kiran Rao)
```